

AI Interview Report

Role: Python Developer

Difficulty: Easy | Interviewer style: Professional

Date: 2026-08-31

Overall score: 8.0 / 10

Filler words used: 1

Summary

The candidate demonstrated a solid grasp of Python fundamentals, including data structures and decorators. Their code quality was generally high, with clear explanations of their thought process. However, the candidate could improve by providing more specific examples and code snippets in their answers.

Q1 [TECHNICAL] Score: 8/10

Write a Python function that takes a list of integers as input and returns the sum of all elements in the list, using a generator expression inside a list comprehension.

Your answer:

You can use `sum(x for x in my_list)` which is a generator expression passed directly to `sum`, avoiding building an intermediate list in memory.

Strengths:

- + Correctly identified the use of generator expressions to avoid memory issues

Suggestions:

- Could provide more context or explanation for why this approach is preferred over other methods

Improved answer:

You can use `sum(x for x in my_list)` which is a generator expression passed directly to `sum`, avoiding the need to build an intermediate list in memory. This approach is memory-efficient, especially for large lists, as it allows the sum calculation to occur on the fly without storing the entire list in memory.

Q2 [BEHAVIORAL] Score: 8/10

Can you describe a situation where you had to use a class to encapsulate data and behavior in a project, and how did you test its functionality?

Your answer:

I built a `Customer` class to encapsulate name, email, and order history. I tested it with `pytest` unit tests covering constructor validation, method behavior, and edge cases like empty order history.

Strengths:

- + Provided specific example of class usage
- + Mentioned testing framework

Suggestions:

- Be more concise
- Explain why you chose `pytest` over other testing frameworks

Improved answer:

I designed a `Customer` class to encapsulate name, email, and order history, and used `pytest` to write unit tests that validated constructor behavior, method functionality, and edge cases like empty order history,

AI Interview Report

allowing me to ensure the class's integrity.

Filler words: 1

Q3 [TECHNICAL] Score: 8/10

Write a Python function that uses a decorator to check if a user is authenticated before allowing access to a sensitive resource.

Your answer:

You write a decorator that wraps the view function, checks `request.user.is_authenticated` (or a session token), and raises a 401 or redirects to login if the check fails, otherwise calls the original function with `functools.wraps` preserving metadata.

Strengths:

- + Uses a decorator to wrap the view function
- + Checks for authentication

Suggestions:

- Use a more secure method to check for authentication, such as verifying a session token
- Explain how to handle the case where the user is authenticated
- Consider using a more robust authentication library

Improved answer:

Use a decorator to wrap the view function, verify a secure session token, and raise a 401 or redirect to login if authentication fails. If authenticated, call the original function with ``functools.wraps`` preserving metadata.

Q4 [PROBLEM-SOLVING] Score: 8/10

Write a Python function called 'most_frequent' that takes a list of strings as input and returns the string that appears most frequently in the list. Use a generator expression to count the frequency of each string, and then use the built-in 'max' function with a key function to find the most frequent string.

Your answer:

`from collections import defaultdict; counts = defaultdict(int); [counts.__setitem__(s, counts[s]+1) for s in items]; return max(counts, key=counts.get).` A cleaner way is `Counter(items).most_common(1)[0][0]`.

Strengths:

- + Used a generator expression to count the frequency of each string
- + Used the max function with a key function to find the most frequent string

Suggestions:

- Could explain the purpose of defaultdict and its usage
- Could provide a more detailed explanation of the Counter class and its advantages

Improved answer:

`from collections import Counter; return Counter(items).most_common(1)[0][0]`